As you can see, the ExampleApp.dll file is still there, along with some additional files. Most notably, the publish process has copied across the wwwroot folder of static files. When running your application locally with `dotnet run`, the .NET CLI uses these files from your application's project folder directly. Running `dotnet publish` copies the files to the output directory, so they're included when you deploy your app to a server.

If your first instinct is to try running the application in the publish folder using the `dotnet run` command you already know and love, you'll be disappointed. Instead of seeing the application starting up, you'll see a somewhat confusing message: `Couldn't find a project to run.`

To run a published application, you need to use a slightly different command. Instead of calling `dotnet run`, you must pass the path to your application's .dll file to the `dotnet` command. If you're running the command from the publish folder, for the example app in figure 27.3, it would look something like

```
dotnet ExampleApp.dll
```

This is the command that your server will run when running your application in production.

TIP You can also use the `dotnet exec` command to achieve the same thing, such as `dotnet exec ExampleApp.dll`. This makes some advanced runtime options available, as described in the docs at http://mng.bz/x4d8.

When you're developing, the `dotnet run` command does a whole load of work to make things easier on you. It makes sure that your application is